MetroInsights Phase 9 - Final Interview Cheat Sheet

High-signal revision bible | Repository revision 4405d167fc323f33be76e2c7f6614fd21cc574a8

5-minute project revision

MetroInsights is a Flask and scikit-learn prototype for station-level passenger-demand scenarios and service planning across a 226-station metro network. A user selects a station, date, time, and (for interchanges) line focus. The backend returns a demand estimate and day profile, date/service context, a schedule recommendation calculated from configured operations assumptions, and estimated network travel times.

The browser calls `/api/predict`. Flask validates date/time syntax, gets station and calendar context, runs the already-trained HistGradientBoostingRegressor using 23 features, calculates line-specific schedules, solves a shortest-path query, and serializes the result as JSON. Forecast/model metadata and compact tables load at startup. `pipeline/` scripts clean data, generate a synthetic training set, train/compare models, and save artifacts. The 130 MB synthetic CSV supports offline retraining; normal inference loads model/metadata instead of reading that CSV each request. Gunicorn starts the Flask app through `Procfile`.

The forecast's inputs include time/date cycles, station/line/typology identifiers, interchange indicators, and historical aggregates. Training compares station-mean and station-day-hour baselines, Ridge, and HistGradientBoostingRegressor using a chronological holdout beginning 2026-10-01. The holdout is still generated by the project's synthetic simulator; metrics are not accuracy against observed DMRC passenger counts.

Scheduling compares estimated hourly demand with configured comfortable boarding-clearance capacity, then returns practical frequency/headway recommendations subject to modeled policy, signaling, and fleet constraints. "Utilization" is not onboard crowding: the system does not model passengers already aboard, station alighting, or train-level occupancy. Travel times use a line-expanded graph and Dijkstra, with coordinate/average-speed edge costs, fallback segment weights, and interchange penalties. They are estimates, not live timetable routes.

The honest framing: this is an end-to-end planning prototype and scenario tool. Its value is the integrated workflow, explicit assumptions, model comparison, and network calculations. Before operational decisions, it needs observed ridership, verified timetables/capacity/topology, domain validation, security/input hardening, tests, and production monitoring.

Project explanations

30 seconds

MetroInsights is a station-level metro demand and service-planning prototype. A user selects a station and scenario time; a Flask API combines a synthetic-data-trained demand model, configurable schedule calculations, calendar context, and Dijkstra-based network travel estimates. It is an interactive planning estimate, not a live DMRC feed or real-world-validated forecast.

2 minutes

The project helps explore how station demand varies by time and how that estimate relates to service recommendations. A browser sends station, date, time, and line-focus inputs to Flask. The API generates a 19-hour station profile from a fitted HistGradientBoostingRegressor, computes schedule recommendations from configurable capacity/headway/fleet assumptions, and returns shortest-path travel-time estimates from a line-expanded metro graph. The offline pipeline prepares station data, generates 104 weeks of synthetic scenarios, compares historical and ML baselines on a chronological holdout, and writes model and metadata artifacts. The web process loads these artifacts once and serves requests; it does not retrain from the 130 MB CSV per user request. Results are useful for demonstrating an integrated workflow, but the synthetic evaluation cannot prove DMRC accuracy. Utilization means estimated hourly demand relative to modeled boarding-clearance capacity, not people physically aboard. The next material step is observed data and expert-verified operating assumptions, followed by time-based validation and safe deployment controls.

One-page architecture map

Layer	Current implementation	Main handoff
Browser	`app/templates/index.html`, `app/static/js/dashboard.js`	POST scenario JSON to `/api/predict`; render response
Web/API	`run.py`, `app/__init__.py`, `app/routes.py`	Validate inputs; orchestrate services; return JSON/HTML
Forecast	`app/forecasting.py`, joblib artifacts	23 features -> hourly/profile demand estimates
Schedule	`app/scheduling_optimizer.py`, operations CSV, line split JSON	station/line demand -> headway, frequency, capacity estimate
Network	`app/network_topology.py`, `app/travel_graph.py`	line topology + weights -> Dijkstra travel-time estimates
Context	`app/calendar_context.py`, `app/operating_hours.py`	date/events and modeled 05:00-23:30 service window
Offline ML	`pipeline/01_*`, `02_*`, `03_*`	source/synthetic data -> cleaned tables -> model/metadata/metrics
Deployment	`Procfile`, `requirements.txt`	`gunicorn run:app`

Request flow: Browser -> `/api/predict` -> validate/date context -> forecast profile -> schedule -> Dijkstra -> JSON -> charts/cards/table. **Training flow:** clean data -> generate synthetic dataset -> chronological train/holdout -> compare models -> save fitted artifacts -> deploy matching code and artifacts.

Important file and function map

File	Know this
`app/routes.py`	`/api/predict`; request orchestration, station/time context, recommendations, travel-time response
`app/forecasting.py`	`predict_hourly_footfall`, minute interpolation, full-day profile, artifact loading
`app/scheduling_optimizer.py`	peak rules, interchange demand split, headway/fleet/capacity recommendations
`app/travel_graph.py`	`build_graph`, `shortest_times_from`, Dijkstra and estimated edge weights
`app/network_topology.py`	line stop order and interchange definitions
`app/calendar_context.py`	event/workday context; events are scenario categories
`app/operating_hours.py`	`is_service_operating`; 05:00 inclusive to 23:30 exclusive
`pipeline/02_generate_calibrated_dataset.py`	104-week synthetic data generation and saved interchange splits
`pipeline/03_train_and_evaluate_forecaster.py`	feature engineering, date split, baselines, training, metrics/artifacts
`models/footfall_forecaster.joblib`	fitted estimator loaded at startup; trust this artifact source
`models/forecaster_metadata.joblib`	feature order, mappings, history aggregates and model context
`models/evaluation_metrics.json`	synthetic holdout metrics, not real-world validation
`data/cleaned/metro_line_operations.csv`	configurable operating assumptions, not verified official schedule data
`app/static/js/dashboard.js`	API fetch, stale-response handling, chart/table rendering

Core algorithms and concepts

• **Feature engineering:** cyclic sin/cos encodings for hour/day/month plus station, line, typology, calendar, and past aggregates.

• **Supervised regression:** HistGradientBoostingRegressor maps 23 feature values to a nonnegative hourly estimate; benchmark it against simple baselines.

• **Chronological holdout:** dates before 2026-10-01 train; later synthetic dates test. This better resembles forecasting forward but remains synthetic.

• **Scheduling arithmetic:** convert service constraints and demand/capacity assumptions into allowable frequency/headway and practical intervals.

• **Graph modeling:** node is `(station, line)`; edges are adjacent-station movement or paid interchange changes.

• **Dijkstra:** priority-queue shortest path for nonnegative weights; returns minimum estimated time, not a real timetable itinerary.

• **Request/response:** JSON API plus browser rendering; input validation and safe text rendering are important boundaries.

Technology decision table

Choice	Why it fits now	Trade-off / alternative
Flask	Small synchronous API and server-rendered entry points	Django/FastAPI may help larger schemas/team needs, with added structure
Pandas/NumPy	Tabular pipeline and feature calculations	Chunking/Polars/SQL if profiling shows data scale pressure
HistGradientBoosting	Nonlinear tabular model with native missing-value support	Seasonal baselines or other models may win with observed data; evaluate, do not assume
Joblib artifacts	Simple local in-process inference	Must trust/version artifact; model service only if lifecycle or load requires it
CSV offline data	Simple reproducible pipeline input/output	Object storage/parquet/database if data access, durability, or parallel processing requires it
Dijkstra	Simple exact shortest path at current graph size	Cache/static precompute first; A* or time-dependent graph for larger/real-time needs
Synchronous request	Easy to reason about for short calculations	Queue jobs only for long-running batch/training/export tasks

Most important implementation details

• Inference uses 23 feature columns specified by saved metadata; deploy model and metadata together.

• Full-day profile covers hours 05:00 through 23:00; the modeled service window is 05:00 inclusive to 23:30 exclusive.

• Unknown station input is not allow-listed in the API and may fall through metadata defaults; reject with 400.

• JSON fields are assumed to have string types; validate types and lengths before `.strip()` or parsing.

• Frontend uses `innerHTML` with dynamic values in travel-table/no-results rendering; render untrusted strings with `textContent`/DOM construction.

• Joblib loading executes a trusted serialization format; never load user-supplied artifacts.

• `dashboard.js` request ID prevents stale responses replacing newer UI content; it does not cancel server-side work.

• `travel_graph` is built at startup; shortest-time searches repeat by origin and could be cached if profiling justifies it.

• Separate Gunicorn worker processes may each hold a model/data copy; tune workers against memory.

• Public endpoints have no app-level authentication or rate limiting in reviewed routes; upstream hosting controls may add boundaries.

• Synthetic holdout metrics, topology, capacity, headway, and interchange shares must not be presented as verified operational truth.

Common interview traps

1. **"Real-time"** - Say "on-demand scenario calculation," not live telemetry. 2. **Accuracy** - Synthetic holdout performance is not real-world DMRC accuracy. 3. **Capacity utilization** - Demand divided by modeled comfortable boarding capacity; not onboard occupancy. 4. **Travel time** - Graph estimate from assumed weights; not live route guidance. 5. **Dataset size** - Millions of synthetic rows do not mean millions of observed passenger records. 6. **"226 stations"** - Network coverage count, not proof every coordinate/topology is authoritative. 7. **Deploying CSV** - The large training CSV is for offline retraining; serving uses model and metadata artifacts. 8. **Scaling** - Measure latency/RSS/CPU before proposing Redis, queues, microservices, or Kubernetes. 9. **Security** - Describe unsafe sinks and preconditions; don't claim a demonstrated remote exploit without proof. 10. **Ownership** - Use only personal contributions and outcomes you can substantiate.

Major limitations

• Training and holdout data are synthetic; no measured DMRC forecast validation.

• Operating capacity, headways, fleet parameters, topology and travel weights may be project assumptions or source data requiring verification.

• No train-level directional load balance, on-board passenger propagation, or boarding/alighting observations.

• Journey estimates omit actual timetables, waiting variability, walking/accessibility, disruptions, and live arrivals.

• Unknown station/wrong-type API inputs need robust validation; request-size/rate controls need deployment review.

• Dynamic frontend `innerHTML` is an unsafe rendering pattern where values are interpolated.

• No comprehensive tracked automated test suite or application monitoring was found in the reviewed repository.

• In-process state/artifacts and worker memory constrain how service behavior scales; no durable user scenario database is present.

Top improvements, in order

1. Obtain observed passenger counts and verified topology/operations data; establish provenance and permissions. 2. Define target meaning and units precisely: station entries, boardings, train load, or another measure. 3. Build time-based evaluation, compare baselines, analyze errors by station/hour/holiday, and document limitations. 4. Validate request schema and station allow-list; bound request sizes; return safe errors. 5. Replace dynamic HTML interpolation with safe DOM/text APIs; protect model artifact supply chain. 6. Add regression tests for service boundaries, unknown/malformed inputs, artifact schema, schedule invariants, and route costs. 7. Measure response latency/RSS/cold starts; batch prediction or precompute route results only if profiling supports it. 8. Add readiness/structured logs/monitoring and a rollbackable code-model release process. 9. Add database, queue, shared cache, or separate model service only when durable product needs or measurements justify them.

Top 100 rapid-fire questions with short answers

• **001. Q:** What is MetroInsights? **A:** A station-level passenger-demand and service-planning prototype.

• **002. Q:** Who is the intended user? **A:** A planner or analyst exploring station/time scenarios; it is not a passenger live-navigation app.

• **003. Q:** What framework serves the app? **A:** Flask, started through Gunicorn.

• **004. Q:** What does the user send? **A:** Station, date, time, and optional line focus.

• **005. Q:** What does the API return? **A:** Forecast profile, context, service recommendation, and estimated travel times.

• **006. Q:** Is it a live passenger feed? **A:** No; it computes on-demand estimates from selected scenario inputs.

• **007. Q:** How many stations are represented? **A:** 226 in the current project network listing.

• **008. Q:** What model is used? **A:** HistGradientBoostingRegressor.

• **009. Q:** How many inference features? **A:** 23, in metadata-defined order.

• **010. Q:** What is the target? **A:** Synthetic station-hour footfall estimate.

• **011. Q:** What is the training source? **A:** A project-generated synthetic dataset.

• **012. Q:** What split is used? **A:** Chronological, with holdout dates from 2026-10-01.

• **013. Q:** Are metrics real-world accuracy? **A:** No, synthetic holdout metrics only.

• **014. Q:** Which baselines are compared? **A:** Station mean, station-day-hour mean, and Ridge regression.

• **015. Q:** Why use cyclical encodings? **A:** They make adjacent cyclic times like 23:00 and 00:00 numerically close.

• **016. Q:** Give an hour encoding example. **A:** Sine and cosine of hour angle.

• **017. Q:** How are features ordered? **A:** Using `feature_cols` from saved metadata.

• **018. Q:** Where is the fitted estimator? **A:** `models/footfall_forecaster.joblib`.

• **019. Q:** Where is feature metadata? **A:** `models/forecaster_metadata.joblib`.

• **020. Q:** Does serving read the large CSV? **A:** The inspected request path loads artifacts and compact tables, not the generated training CSV.

• **021. Q:** What is the CSV for? **A:** Offline training/retraining and pipeline data preparation.

• **022. Q:** What hours are charted? **A:** 05:00 through 23:00 hourly points.

• **023. Q:** When does modeled service begin? **A:** 05:00 inclusive.

• **024. Q:** When does modeled service end? **A:** 23:30 exclusive.

• **025. Q:** What happens when closed? **A:** Demand and recommended trains are set to zero/closed state.

• **026. Q:** What is capacity utilization? **A:** Estimated hourly demand divided by modeled comfortable boarding-clearance capacity.

• **027. Q:** Does it equal onboard occupancy? **A:** No.

• **028. Q:** What is missing for onboard load? **A:** Directional boardings/alightings and load carried between stations.

• **029. Q:** How are schedules recommended? **A:** By applying configurable demand, capacity, headway, fleet, and policy assumptions.

• **030. Q:** Are those assumptions official? **A:** They require verification; provenance labels mark configurable assumptions.

• **031. Q:** What is an interchange line split? **A:** A modeled allocation of station demand across connected lines.

• **032. Q:** Is the line split observed? **A:** No, it is a synthetic weighted estimate or fallback.

• **033. Q:** How is the graph node defined? **A:** A `(station, line)` pair.

• **034. Q:** Why line-expanded nodes? **A:** To charge a cost when the route changes lines at an interchange.

• **035. Q:** Which shortest-path algorithm? **A:** Dijkstra.

• **036. Q:** Why Dijkstra? **A:** All edge costs are nonnegative and the current graph is small.

• **037. Q:** Are edge costs live? **A:** No; they use estimates/assumptions.

• **038. Q:** What if coordinates are missing? **A:** The graph uses a default segment-time fallback.

• **039. Q:** Does a route include train waiting? **A:** Not as a live scheduled wait; it is an estimated graph travel time.

• **040. Q:** Where is topology defined? **A:** `app/network_topology.py`.

• **041. Q:** Where is graph search? **A:** `app/travel_graph.py`.

• **042. Q:** Where is inference? **A:** `app/forecasting.py`.

• **043. Q:** Where are schedule rules? **A:** `app/scheduling_optimizer.py`.

• **044. Q:** Where is calendar context? **A:** `app/calendar_context.py`.

• **045. Q:** Where is service-window logic? **A:** `app/operating_hours.py`.

• **046. Q:** Where is client fetch/render logic? **A:** `app/static/js/dashboard.js`.

• **047. Q:** What loads at app startup? **A:** Model/metadata, station/operations tables, and the route graph.

• **048. Q:** What runs per prediction request? **A:** Forecast, schedule calculation, graph search, and response serialization.

• **049. Q:** Does the browser request ID cancel work? **A:** No, it only ignores stale responses in the UI.

• **050. Q:** What does `tojson` help protect? **A:** Safe serialization of server-provided station metadata into JavaScript context.

• **051. Q:** What frontend pattern needs improvement? **A:** Dynamic strings inserted into `innerHTML`.

• **052. Q:** What should replace interpolated text? **A:** DOM creation and `textContent`.

• **053. Q:** Is there authentication? **A:** No app-level authentication was present in reviewed routes.

• **054. Q:** Are there app-level rate limits? **A:** None were found in the reviewed app code.

• **055. Q:** Should this public endpoint get body limits? **A:** Yes, configure a reasonable maximum and validate fields.

• **056. Q:** What response for an unknown station? **A:** A controlled 400 client error after allow-list validation.

• **057. Q:** What is wrong with fallback station index? **A:** It may return a plausible estimate for the wrong station.

• **058. Q:** What file starts the service on Render? **A:** `Procfile`, with `gunicorn run:app`.

• **059. Q:** Should training run during a web request? **A:** No, keep training in an offline pipeline/job.

• **060. Q:** What is the first performance step? **A:** Measure cold/warm latency, CPU, memory, and route-stage timings.

• **061. Q:** What does p95 mean? **A:** 95 percent of measured requests completed within that duration.

• **062. Q:** Could batch inference help? **A:** Possibly; compare 19 one-row calls with one matrix call and verify parity.

• **063. Q:** Could route caching help? **A:** Possibly; static topology makes origin results deterministic per version.

• **064. Q:** What must a forecast cache key include? **A:** Station, date, model/data/calendar version, and other response-affecting inputs.

• **065. Q:** Should every response be cached? **A:** No; cache stable subresults and bound/invalidate them correctly.

• **066. Q:** What is a Gunicorn worker? **A:** A process or worker handling web requests.

• **067. Q:** Why can more workers use more RAM? **A:** Each process may load its own libraries, model, and tables.

• **068. Q:** When add a queue? **A:** For long-running or asynchronous batch work.

• **069. Q:** When add a database? **A:** When durable scenarios, observations, or audit history are needed.

• **070. Q:** When split a model service? **A:** When independent lifecycle, load, or dependencies justify extra complexity.

• **071. Q:** What is the first scale-out option? **A:** Stateless web instances behind a load balancer, after measuring saturation.

• **072. Q:** What does the health check verify? **A:** That required artifacts loaded and the service can respond minimally.

• **073. Q:** Why version artifacts? **A:** To keep estimator, features, and code compatible and rollback together.

• **074. Q:** What is a serialized artifact risk? **A:** Loading an untrusted joblib/pickle artifact can execute code.

• **075. Q:** Where should training data live? **A:** In controlled durable artifact/object storage if reproducibility or size requires it.

• **076. Q:** What does reproducibility need? **A:** Data/code/version provenance, seeds, dependency versions, and artifact identity.

• **077. Q:** What does a fixed random seed ensure? **A:** Repeatable random-number sequence under compatible execution, not real-world validity.

• **078. Q:** What does MAPE struggle with? **A:** Targets near or equal to zero.

• **079. Q:** Why compare a simple baseline? **A:** To show whether complexity adds predictive value.

• **080. Q:** What does chronological validation simulate? **A:** Using past data to predict a later time period.

• **081. Q:** Can synthetic holdout prove deployment accuracy? **A:** No.

• **082. Q:** How should a forecast be described? **A:** As a synthetic-data-trained model estimate for a scenario.

• **083. Q:** How to handle null station JSON? **A:** Reject it with schema validation before calling string methods.

• **084. Q:** Which errors does current date parsing catch? **A:** Malformed date/time `ValueError` cases.

• **085. Q:** Does parsing constrain the date range? **A:** No; parseable dates outside the validated horizon can be requested.

• **086. Q:** What should distant dates do? **A:** Be rejected or visibly marked out-of-domain.

• **087. Q:** What if route search fails? **A:** Return clear partial/unavailable route status if that section is nonessential.

• **088. Q:** What if model metadata mismatches? **A:** Fail readiness/startup clearly instead of serving corrupt results.

• **089. Q:** What should tests cover first? **A:** Input schema, station allow-list, service boundaries, artifact schema, schedule invariants.

• **090. Q:** How to test availability safely? **A:** Use local/disposable bounded requests, not a production flood.

• **091. Q:** What does privacy require here? **A:** Minimize user-linked data and define access/retention if scope expands.

• **092. Q:** Does current flow appear to store personal passenger data? **A:** No passenger-level records were evident in the reviewed request flow.

• **093. Q:** What does p50 represent? **A:** Median measured latency.

• **094. Q:** What is cold-start latency? **A:** Startup/wake-up delay before normal warm request behavior.

• **095. Q:** What is graceful degradation? **A:** Keep core useful output available when a nonessential subcomponent fails, with clear status.

• **096. Q:** Why avoid microservices now? **A:** Current scale does not justify their operational overhead without evidence.

• **097. Q:** What is one data improvement? **A:** Obtain observed, governed ridership counts with clear target definition.

• **098. Q:** What is one domain improvement? **A:** Verify actual timetable, capacity, topology, and station/direction definitions.

• **099. Q:** What personal claims belong on a resume? **A:** Only contributions and outcomes that can be substantiated.

• **100. Q:** Best answer to "is it production ready?" **A:** It is a prototype; observed-data validation and reliability/security work remain.

Top 50 difficult questions with concise answers

• **01. Q:** How would you prove this model is useful beyond synthetic data? **A:** Obtain governed observed counts, define the target, use temporal holdouts, compare baselines, stratify errors, and get domain review.

• **02. Q:** Why can a chronological split still overstate future performance? **A:** The synthetic generator may encode the same assumptions in train and holdout; shared simulator structure is not independent real-world evidence.

• **03. Q:** Could the training process leak a row's target into its historical features? **A:** The current rolling aggregates subtract the current target and use prior dates; audit every future feature for prediction-time availability.

• **04. Q:** Why are per-row training history features different from live history metadata? **A:** They are constructed on separate paths; verify identical definitions, fallback behavior, and feature distributions to avoid train/serve skew.

• **05. Q:** Why not use random train/test splitting? **A:** Temporal dependencies can place near-identical dates in both sets and make future generalization look easier than it is.

• **06. Q:** What does R-squared fail to tell you here? **A:** It does not prove real-world validity, reveal station-specific failures, or describe absolute error and operational cost.

• **07. Q:** Why is MAPE problematic for low/zero footfall? **A:** Division by small actual values creates unstable or undefined percentage errors; complement it with MAE/RMSE and segmented analysis.

• **08. Q:** How do you detect drift once observed data exists? **A:** Track feature/target distributions and rolling time-based residuals by station, line, hour, and event; define retraining and escalation criteria.

• **09. Q:** How do you prevent future information in a calendar feature? **A:** Version the calendar source and include only information known at the selected forecast date; distinguish announced events from realized outcomes.

• **10. Q:** What if a station identifier appears during inference but was unseen in training? **A:** Use a deliberate unknown-category policy, report it, and validate coverage; do not silently map it to a real station index.

• **11. Q:** Why might an ordinal station index be a questionable model feature? **A:** The numeric order has no inherent distance meaning; tree splits may exploit arbitrary category ordering. Compare one-hot, categorical-aware, or embedding alternatives.

• **12. Q:** Would one global model or a station-specific model be better? **A:** Compare on temporal validation: a global model shares strength, while local models capture station patterns but need sufficient data and maintenance.

• **13. Q:** How should uncertainty be represented? **A:** Calibrate prediction intervals or quantiles on observed time-separated residuals; a point estimate alone hides uncertainty.

• **14. Q:** What does station-hour footfall mean operationally? **A:** Its precise definition must be fixed: entries, boardings, exits, or another count. The current synthetic target is not automatically onboard load.

• **15. Q:** Why can summed station demand double-count network riders? **A:** A rider may enter/exit at different stations or be counted under the synthetic target definition; network totals require a clear counting unit.

• **16. Q:** How would you calculate train onboard occupancy? **A:** Propagate directional passenger loads segment by segment using boardings, alightings, train assignments, and capacity, then validate measured loads.

• **17. Q:** Why is capacity utilization not a queueing model? **A:** It compares hourly demand with modeled hourly clearance capacity; it lacks arrival distributions, dwell time, queue state, and train-by-train loads.

• **18. Q:** What happens if demand exceeds the modeled fleet maximum? **A:** The optimizer should expose unmet demand/constraint status; verify calculations and never imply a feasible schedule if constraints bind.

• **19. Q:** How do you test a headway rounding invariant? **A:** Generate boundary cases and assert rounded values remain within signaling, policy, and fleet-derived frequency bounds.

• **20. Q:** Why could rounded headway violate a fleet constraint? **A:** Selecting the nearest practical headway after frequency clamping can alter frequency; recompute all constraints after rounding.

• **21. Q:** What does Dijkstra guarantee? **A:** A minimum summed path cost when edge weights are nonnegative; it does not guarantee that weights model reality.

• **22. Q:** Why use `(station, line)` rather than station-only graph nodes? **A:** It encodes platform/line state so line changes incur explicit interchange edges.

• **23. Q:** What breaks if interchange edges are missing? **A:** Routes that require transfers may become unreachable or incorrectly disconnected.

• **24. Q:** What breaks if topology says a false adjacency? **A:** Dijkstra can return a mathematically shortest but physically invalid route; topology source validation is essential.

• **25. Q:** How would you replace average-speed times with realistic journey times? **A:** Use verified segment running/dwell times, direction, service patterns, transfer walking/waiting and schedule/real-time data.

• **26. Q:** Would A* be faster than Dijkstra on this app? **A:** Possibly for large graphs with an admissible heuristic; at a few hundred stations, simpler Dijkstra may be adequate.

• **27. Q:** What is safe to cache in route calculations? **A:** Origin-to-destination results for a versioned immutable graph; invalidate on topology or weight changes.

• **28. Q:** What would you put in a forecast cache key? **A:** Canonical station, date, model/data/calendar versions, feature schema, and every input affecting the profile.

• **29. Q:** Why must an in-memory cache be bounded? **A:** Arbitrary station/date keys could grow process memory without limit.

• **30. Q:** Why is a per-worker cache not shared? **A:** Gunicorn workers are separate processes with separate heaps; sharing needs external storage or precomputed artifacts.

• **31. Q:** How would you avoid train/serve skew? **A:** Use one tested feature builder for training/inference where possible and validate schema/value parity with fixed fixtures.

• **32. Q:** How can you safely roll out a new model? **A:** Build immutable versioned artifacts, validate schema and smoke predictions, deploy code/artifact together, monitor, then retain rollback.

• **33. Q:** Why is joblib a security boundary? **A:** It deserializes Python objects; only load trusted, integrity-checked artifacts.

• **34. Q:** What checks belong in readiness? **A:** Required artifact presence, hashes/schema compatibility, and a minimal safe inference/graph check.

• **35. Q:** What is the difference between readiness and liveness? **A:** Readiness says traffic can be served; liveness says the process is alive and should not be restarted.

• **36. Q:** How do you bound request resource use? **A:** Cap body size and field lengths, validate type/allowed values, rate-limit upstream, and set concurrency/timeouts.

• **37. Q:** Why is a wrong JSON type more than a UX issue? **A:** It can trigger unhandled exceptions and 500s, increasing attack surface and reducing availability.

• **38. Q:** What is the DOM XSS concern and limitation? **A:** Dynamic `innerHTML` is an unsafe sink; current exploitability depends on attacker control of the interpolated data path.

• **39. Q:** Why does `tojson` not sanitize all JavaScript rendering? **A:** It protects that serialized template context; later HTML string interpolation creates independent sinks.

• **40. Q:** When is CSRF relevant? **A:** When browser credentials/cookies authorize state-changing actions; current prediction endpoint is computation-only, but future writes need protection.

• **41. Q:** Why may adding auth be the wrong immediate fix? **A:** The reviewed app is a public read-only demo; first identify protected assets and access requirements, then add proportionate controls.

• **42. Q:** How do you protect privacy if observed data arrives? **A:** Minimize and aggregate, assess re-identification risk, control access/retention, document permissions, and avoid exposing row-level data.

• **43. Q:** Why pin the Python dependency environment? **A:** Repeatable builds and model artifact compatibility; maintain a reviewed lock/constraints file with intentional updates.

• **44. Q:** What if scikit-learn upgrades but the artifact stays old? **A:** Loading may fail or behave incompatibly; test exact runtime versions and redeploy compatible artifact/software as a pair.

• **45. Q:** What can cause cold starts? **A:** Platform sleep/wake, Python imports, model loading, and graph construction; measure these separately from warm request latency.

• **46. Q:** How do workers trade memory for throughput? **A:** More processes can handle parallel requests but each may duplicate model/runtime memory and increase contention.

• **47. Q:** When is async job processing appropriate? **A:** For long batch forecasts, exports, or training; it requires durable queue/status, retries, idempotency, result storage, and monitoring.

• **48. Q:** How do you choose database versus object storage? **A:** Database for queried structured durable records; object storage for large immutable datasets/artifacts; cache for recomputable values.

• **49. Q:** What are five useful production signals? **A:** p95 latency, error/timeout rate, CPU, RSS/restarts, and cold-start/deploy health, segmented by route/revision.

• **50. Q:** How do you test a deployment without making risky changes? **A:** Use a staging or disposable instance, smoke-test startup and representative requests, check logs/health, then promote with rollback ready.

One-page printable cheat sheet

Purpose: Print this page and use it for a final 60-second scan.

What it is: MetroInsights is a Flask/scikit-learn station-level demand and service-planning prototype covering 226 stations. User selects station/date/time/line; API returns a scenario forecast, schedule estimate, calendar/service context, and graph travel-time estimates.

Request: Browser `dashboard.js` -> `POST /api/predict` in `routes.py` -> validate/context -> `forecasting.py` -> `scheduling_optimizer.py` -> `travel_graph.py` Dijkstra -> JSON -> UI.

Model: HistGradientBoostingRegressor; 23 metadata-ordered features; compared with station mean, station-day-hour mean, and Ridge; chronological holdout from 2026-10-01.

Data truth: Synthetic simulation data; not observed DMRC footfall. 130 MB CSV is for offline training; inference loads model and metadata artifacts.

Capacity truth: Demand / modeled comfortable hourly boarding-clearance capacity. Not train onboard occupancy; no onboard load propagation or observed board/alight counts.

Travel truth: `(station, line)` graph, adjacent-station weights + interchange penalty, Dijkstra. Estimates, not live itinerary/timetable.

Core files: routes, forecasting, scheduling_optimizer, travel_graph, network_topology, calendar_context, operating_hours; pipelines 01 clean, 02 synthesize, 03 train/evaluate.

Say precisely: "On-demand scenario estimate," "synthetic holdout metrics," "configurable operations assumptions," "estimated travel time."

Do not claim: live passenger feed, real-world accuracy, official verified capacity, onboard crowding, exact live travel time, measured operational impact.

First improvements: observed/verified data + defined target; temporal validation; station/type validation; safe DOM text rendering; body/rate limits; regression tests; artifact versioning; monitoring.

Scale order: measure p95/CPU/RSS/cold starts -> batch profile or cache static routes if profiled -> separate offline jobs -> add data/cache/services only for proven needs.

Strong answer pattern: Direct answer -> code evidence -> assumption/limitation -> how to verify/improve.

Three defensible admissions: No measured DMRC accuracy yet. Utilization is not occupancy. This is a prototype, not an operations control system.

Ultra-short final revision

Prototype, not live DMRC operations. Synthetic-data-trained HistGradientBoosting model, 23 features, chronological synthetic holdout. Flask `/api/predict` orchestrates forecast + schedule assumptions + Dijkstra graph estimates. Large CSV is offline training input; web serving loads model artifacts. Utilization means estimated demand over modeled boarding capacity, not people onboard. Travel times are estimates. Validate with observed data, harden inputs/rendering, test invariants, version artifacts, measure before scaling.